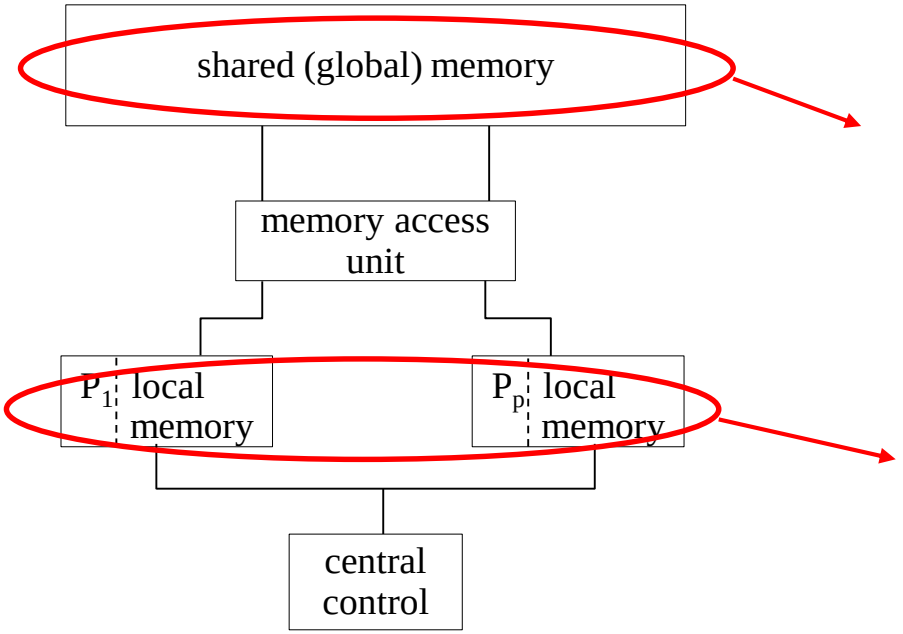
PRAM

- Parallel random access machine



the only way of communication

At each step, all the processors execute the phases:

- read phase
- compute phase
- write phase

- Built up to 512, 1024, etc. processors
- Technological limitations in processor-memory connections

PRAM

- A good model for studying parallel computation
  - Limits of parallel computation

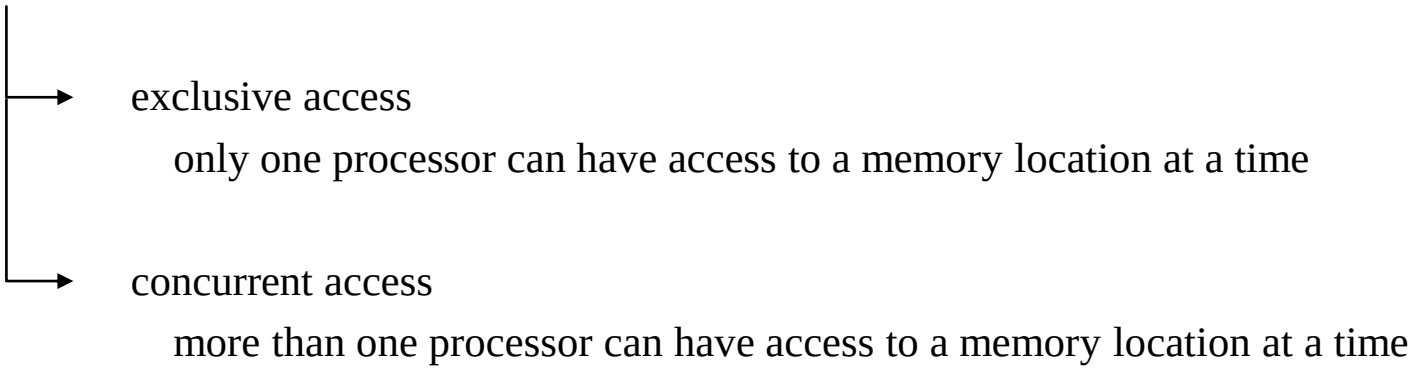
    The asymptotic running time of an optimal PRAM algorithm is at least as fast as that of an optimal algorithm on any other architecture with the same number of processors

    P_i and P_j communication $\rightarrow$ 1 step – $\theta(1)$ time

  - Focus on the potential parallelism in a problem

PRAM – Data access conflicts

- Data access conflicts
  - Processors accessing the “same memory location” simultaneously
  - Reading & Writing



PRAM – Data access conflicts

- EREW (exclusive read, exclusive write)
  - Most restrictive model
  - Most realistic model
    Building processor–memory communications
  - More difficult for algorithm design
    Not to violate ER/EW restriction
  - Can execute on other PRAM models
    But, probably will not be optimal

PRAM – Data access conflicts

- CREW (concurrent read, exclusive write)
- ERCW (exclusive read, concurrent write)
  - Makes sense?
- CRCW (concurrent read, concurrent write)
  - Most flexible model
  - How to resolve write conflicts?

PRAM – Data access conflicts

- CRCW (concurrent read, concurrent write)
  - Priority CW

    The processor with the highest priority is allowed to write
  - Common CW

    Write operation is allowed only if processors attempt to write the same data
  - Arbitrary CW

    A processor chosen randomly is allowed to write
  - Combining CW

    Combined value is written

    → arithmetic functions (sum, product, …)
    logical functions (and, or, xor, …)
    higher-level operations (max, min, …)

Are all useful?
e.g. Where to use arbitrary CW?

PRAM – Broadcasting

```
function BroadcastEREW (A[0:n-1], X)
Model:   EREW PRAM with p=n/2 processors
Input:   A[0:n-1] (an array of size n, n=2^k)
         X (a value to be broadcast throughout A[0:n-1])
Output:  A[0:n-1] (an array where A[i]=X, 0≤i≤n-1)

   A[0] ← X
   for i ← 1 to log2n do
      for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
         A[j] ← A[j-2^(i-1)]
      end in parallel
   endfor
end
```

Follow
our pseudocode convention
for sequential & parallel algorithms
Textbook, Appendix G

Broadcasting:
A piece of data that resides in one of the processors or in shared memory is distributed to an array in shared memory

Initial operation in PRAM algorithms

PRAM – Broadcasting

```
function BroadcastEREW (A[0:n-1], X)
Model:   EREW PRAM with p=n/2 processors
Input:   A[0:n-1] (an array of size n, n=2^k)
         X (a value to be broadcast throughout A[0:n-1])
Output:  A[0:n-1] (an array where A[i]=X, 0≤i≤n-1)

   A[0] ← X
   for i ← 1 to log2n do
      for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
         A[j] ← A[j-2^(i-1)]
      end in parallel
   endfor
end
```

heading part

function <module name><PRAM type> (<parameters>)
Model: <model name> with p=<pr no> processors
Input: <explanation of input parameters>
Output: <explanation of output parameters>

PRAM – Broadcasting

- For … in parallel:
  - Syntax:

```
for <expression with array indices> do in parallel
    <statement_1>
    <statement_2>
    …
    <statement_n>
end in parallel
```

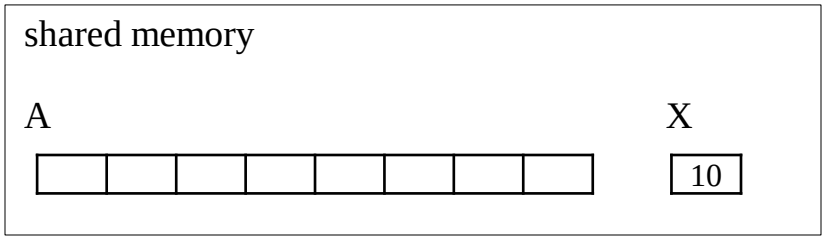
```
function BroadcastEREW (A[0:n-1], X)
Model:   EREW PRAM with p=n/2 processors
Input:   A[0:n-1] (an array of size n, n=2^k)
         X (a value to be broadcast throughout A[0:n-1])
Output:  A[0:n-1] (an array where A[i]=X, 0≤i≤n-1)

  A[0] ← X
  for j ← 1 to log2n do
    for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
      A[j] ← A[j-2^(i-1)]
    end in parallel
  endfor
end
```

  - Parallelism is achieved by the use of arrays
  - Loop expression shows how many processors are active
    - Which processors? ✗
    - Be careful ≤ number of processors
  - Statements are executed by that number of processors
    - Each value of loop control variable → one execution

PRAM – Broadcasting

- Execution (n=8)

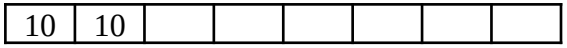


function BroadcastEREW (A[0:n-1], X)
Model: EREW PRAM with p=n/2 processors
Input: A[0:n-1] (an array of size n, $n=2^k$)
X (a value to be broadcast throughout A[0:n-1])
Output: A[0:n-1] (an array where A[i]=X, $0 \le i \le n-1$)

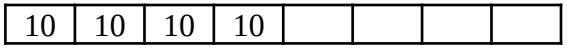
```
A[0] ← X
for i ← 1 to log2n do
   for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
      A[j] ← A[j-2^(i-1)]
   end in parallel
endfor
end
```

A[0] ← X ⇒ 

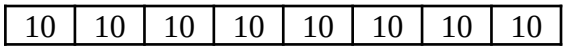
P_1: A[0]←X

i=1
for 1 ≤ j ≤ 1 **do in parallel** ⇒ 

P_1: A[1]←A[0]

i=2
for 2 ≤ j ≤ 3 **do in parallel** ⇒ 

P_1: A[2]←A[0]
P_2: A[3]←A[1]

i=3
for 4 ≤ j ≤ 7 **do in parallel** ⇒ 

P_1: A[4]←A[0]
P_2: A[5]←A[1]
P_3: A[6]←A[2]
P_4: A[7]←A[3]

PRAM – Broadcasting

```
function BroadcastEREW (A[0:n-1], X)
Model:   EREW PRAM with p=n/2 processors
Input:   A[0:n-1] (an array of size n, n=2^k)
         X (a value to be broadcast throughout A[0:n-1])
Output:  A[0:n-1] (an array where A[i]=X, 0≤i≤n-1)

   A[0] ← X
   for i ← 1 to log2n do
      for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
         A[j] ← A[j-2^(i-1)]
      end in parallel
   endfor
end
```

```
k ← 1
for i ← 1 to log2n do
   for k ≤ j ≤ 2*k-1 do in parallel
      A[j] ← A[j-k]
   end in parallel
   k ← 2*k
endfor
```

PRAM – Broadcasting

function BroadcastEREW (A[0:n-1], X)
Model: EREW PRAM with p=n/2 processors
Input: A[0:n-1] (an array of size n, $n=2^k$)
X (a value to be broadcast throughout A[0:n-1])
Output: A[0:n-1] (an array where A[i]=X, $0 \le i \le n-1$)

```
A[0] ← X
for i ← 1 to log2n do
   for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
      A[j] ← A[j-2^(i-1)]
   end in parallel
endfor
end
```

function BroadcastCRCW (A[0:n-1], X)
Model: CRCW PRAM with p=n processors
Input: A[0:n-1] (an array of size n, $n=2^k$)
X (a value to be broadcast throughout A[0:n-1])
Output: A[0:n-1] (an array where A[i]=X, $0 \le i \le n-1$)

end

PRAM – Broadcasting

- Complexity analysis
  - General rule for basic operation



parallel operation
+
sequential work (*rare*)

```
function BroadcastEREW (A[0:n-1], X)
Model:   EREW PRAM with p=n/2 processors
Input:   A[0:n-1] (an array of size n, n=2^k)
         X (a value to be broadcast throughout A[0:n-1])
Output:  A[0:n-1] (an array where A[i]=X, 0≤i≤n-1)

  A[0] ← X
  for i ← 1 to log2n do
    for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
      A[j] ← A[j-2^(i-1)]
    end in parallel
  endfor
end
```

PRAM – Broadcasting

- Complexity analysis
  - Basic operation: parallel assignment statement

function BroadcastEREW (A[0:n-1], X)
Model: EREW PRAM with p=n/2 processors
Input: A[0:n-1] (an array of size n, $n=2^k$)
X (a value to be broadcast throughout A[0:n-1])
Output: A[0:n-1] (an array where A[i]=X, $0 \le i \le n-1$)

```
A[0] ← X
for i ← 1 to log2n do
   for 2^(i-1) ≤ j ≤ 2^i -1 do in parallel
      A[j] ← A[j-2^(i-1)]
   end in parallel
endfor
end
```

1

$\log_2 n$

$$\Rightarrow A(n) = B(n) = W(n) = \log_2 n \in \theta(\log n)$$

Speedup: $\frac{n}{\log n}$

PRAM – Broadcasting

- What is the problem here?

```
for 0 ≤ i ≤ n-1 do in parallel
   if i ≤ 10 then
      A[i] ← 2*A[i]
   else
      A[i] ← A[i]+1
   endif
end in parallel
```

Violates SIMD model of computation

```
for 0 ≤ i ≤ 10 do in parallel
      A[i] ← 2*A[i]
end in parallel

for 11 ≤ i ≤ n-1 do in parallel
      A[i] ← A[i]+1
end in parallel
```

2 parallel steps instead of 1 parallel step

PRAM – Minimum Finding

- Semigroup operation $\otimes$

  An associate binary operation – $(x \otimes y) \otimes z = x \otimes (y \otimes z)$

  - Example
    - Maximum
    - Minimum
    - Addition
    - Multiplication
    - Logical and
    - …

PRAM – Minimum Finding

```
function MinEREW (L[0:n-1])
Model:   EREW PRAM with p=n/2 processors
Input:   L[0:n-1] (a list of size n, n=2^k)
Output:  minimum of the elements in L

   for i ← 1 to log2n do
      for 0 ≤ j ≤ n/2^i − 1 do in parallel
         if L[2j+1] > L[2j] then
            L[j] ← L[2j]
         else
            L[j] ← L[2j+1]
         endif
      end in parallel
   endfor
   return (L[0])
end
```

PRAM – Minimum Finding

- Execution (n=8)

shared memory

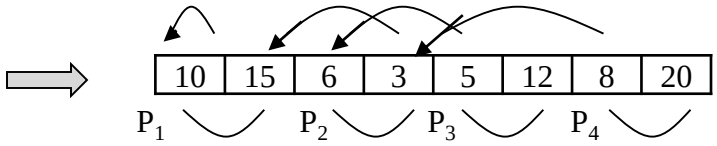
L

10	15	6	3	5	12	8	20

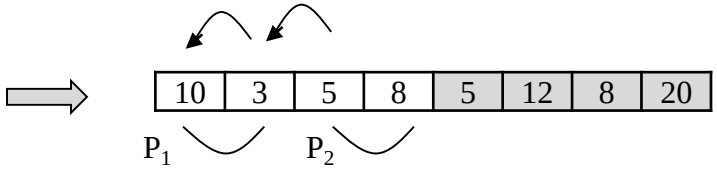
```
function MinEREW (L[0:n-1])
Model:  EREW PRAM with p=n/2 processors
Input:  L[0:n-1] (a list of size n, n=2^k)
Output: minimum of the elements in L

  for i ← 1 to log2n do
    for 0 ≤ j ≤ n/2^i − 1 do in parallel
      if L[2j+1] > L[2j] then
        L[j] ← L[2j]
      else
        L[j] ← L[2j+1]
      endif
    end in parallel
  endfor
  return (L[0])
end
```

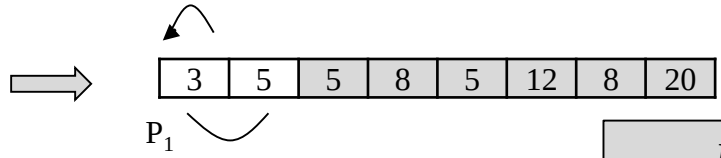
i=1
for 0 ≤ j ≤ 3 **do in parallel**

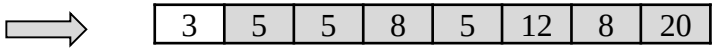


i=2
for 0 ≤ j ≤ 1 **do in parallel**



i=3
for 0 ≤ j ≤ 0 **do in parallel**





Binary fan-in

PRAM – Minimum Finding

- Complexity analysis
  - Basic operation: parallel comparison

```
function MinEREW (L[0:n-1])
Model:   EREW PRAM with p=n/2 processors
Input:   L[0:n-1] (a list of size n, n=2^k)
Output:  minimum of the elements in L

   for i ← 1 to log2n do
      for 0 ≤ j ≤ n/2^i − 1 do in parallel
         if L[2j+1] > L[2j] then
            L[j] ← L[2j]
         else
            L[j] ← L[2j+1]
         endif
      end in parallel
   endfor
   return (L[0])
end
```

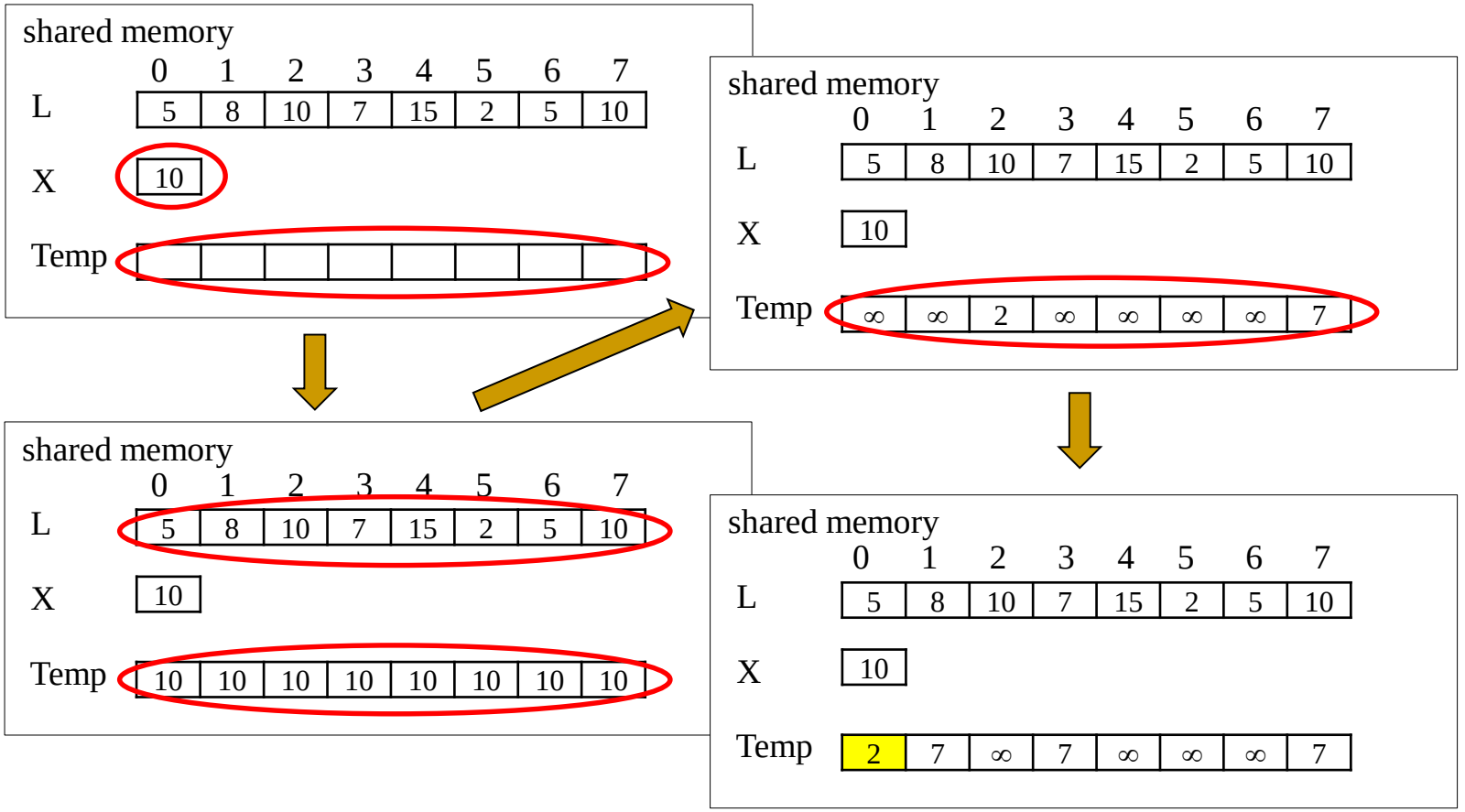
1

$\log_2 n$

$\Rightarrow A(n) = B(n) = W(n) = \log_2 n \in \theta(\log n)$

Speedup: $\frac{n}{\log n}$

PRAM – Searching



PRAM – Searching

```
function SearchEREW (L[0:n-1], X)
Model:   EREW PRAM with p=n processors
Input:   L[0:n-1] (a list of size n, n=2^k)
         X (a search element)
Output:  the smallest index where X occurs in L, or ∞ if X is not in L

   call BroadcastEREW (Temp[0:n-1], X)

   for 0 ≤ i ≤ n-1 do in parallel
      if L[i] = Temp[i] then
         Temp[i] ← i
      else
         Temp[i] ← ∞
      endif
   end in parallel

   return (MinEREW (Temp[0:n-1]))
end
```

PRAM – Searching

- Complexity analysis
  - Basic operation: parallel assignments, parallel comparisons, parallel comparisons

```
function SearchEREW (L[0:n-1], X)
Model:   EREW PRAM with p=n processors
Input:   L[0:n-1] (a list of size n, n=2^k)
         X (a search element)
Output:  the smallest index where X occurs in L, or ∞ if X is not in L
   call BroadcastEREW (Temp[0:n-1], X)
   for 0 ≤ i ≤ n-1 do in parallel
      if L[i] = Temp[i] then
         Temp[i] ← i
      else
         Temp[i] ← ∞
      endif
   end in parallel
   return (MinEREW (Temp[0:n-1]))
end
```

$\log_2 n$

1

$\log_2 n$

Not suitable as the single operation

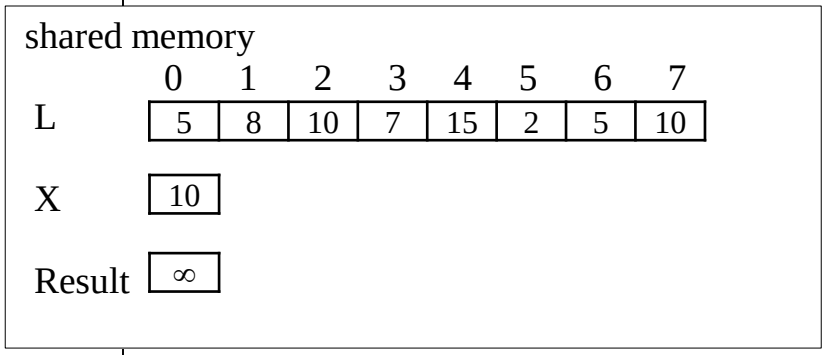
$\Rightarrow A(n) = B(n) = W(n) = 2\log_2 n + 1 \in \theta(\log n)$

PRAM – Searching

```
function SearchEREW (L[0:n-1], X)
Model:   EREW PRAM with p=n processors
   call BroadcastEREW (Temp[0:n-1], X)

   for 0 ≤ i ≤ n-1 do in parallel
      if L[i] = Temp[i] then
         Temp[i] ← i
      else
         Temp[i] ← ∞
      endif
   end in parallel

   return (MinEREW (Temp[0:n-1]))
end
```




```
function SearchCRCW (L[0:n-1], X)
Model:   CRCW PRAM with p=n processors




end
```

PRAM – Searching

```
function SearchCRCW (L[0:n-1], X)
Model:   CRCW PRAM with p=n processors

   for 0 ≤ i ≤ n-1 do in parallel
      if L[i] = X then
         Result ← i
      endif
   end in parallel
end
```

- Involves CR?
- Involves CW?

  Which CW mechanism?

PRAM – Goodness Criteria

- How much time we gain?
  - Example: Minimum finding

Sequential algorithm	Parallel algorithm	Speedup
$\theta(n)$	$\theta(\log n)$	$\frac{n}{\log n}$

  - Example: Searching

e.g. n=1000 $\Rightarrow$ 100 times faster

Sequential algorithm	Parallel algorithm	Speedup
$\theta(n)$	$\theta(\log n)$	$\frac{n}{\log n}$

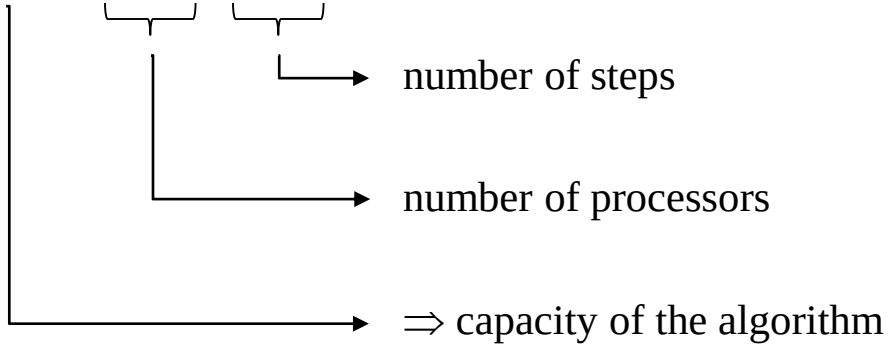
Good parallel algorithms?

PRAM – Goodness Criteria

- Terminology
  - $W(n)$: Worst-case complexity of a parallel algorithm for a data size n
    Traditionally, we use $W(n)$
    Mostly, $W(n) = A(n) = B(n)$
  - $p(n)$: Number of processors used in a parallel algorithm
  - $W^*(n)$: Worst-case complexity of the optimal (or, best) sequential algorithm for a problem

PRAM – Goodness Criteria

- Cost of a parallel algorithm
  - $C(n) = p(n) * W(n)$



PRAM – Goodness Criteria

- Cost of a parallel algorithm
  - $C(n) = p(n) * W(n)$
- How to evaluate cost?
  - For a problem:

    Can be solved using a sequential algorithm with $W^*(n)$ basic operations (steps/time/ complexity)

    We have $p(n)$ processors

    Can be solved using a "good" parallel algorithm in $\frac{W^*(n)}{p(n)}$ steps (time/complexity)

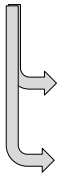
    That is, $W(n) = \frac{W^*(n)}{p(n)} \Rightarrow W(n) * p(n) = W^*(n)$

    $\Rightarrow C(n) = W^*(n)$

If $W^*(n)$ is the best we can do sequentially, then $\frac{W^*(n)}{p(n)}$ is the best for the parallel solution of that problem

PRAM – Goodness Criteria

- Goodness of parallel algorithms

  wrt time criterion

  wrt cost criterion

- A parallel algorithm is

  - Cost optimal: $C(n) = W^*(n)$

  - Cost efficient: $C(n)$ is within a polylogarithmic factor of being cost optimal

    i.e. $C(n) \in O\left(W^*(n) * log^k W^*(n)\right)$, $k \geq 0$

  - Time efficient: $W(n)$ is polylogarithmic

    i.e. $W(n) \in O\left(log^k n\right)$, $k \geq 0$

PRAM – Goodness Criteria

- A parallel algorithm is
  - Cost optimal: $C(n) = W^*(n)$
  - Cost efficient: $C(n)$ is within a polylogarithmic factor of being cost optimal
    i.e. $C(n) \in O\left(W^*(n) * log^k W^*(n)\right), k \geq 0$
  - Time efficient: $W(n)$ is polylogarithmic
    i.e. $W(n) \in O\left(log^k n\right), k \geq 0$

Are the processors utilized well?

Is it feasible to execute the algorithm?

$O(n)$, $O\left(n^k\right)$ not feasible ???

PRAM – Goodness Criteria

- For a parallel algorithm
  - Speedup: $S(n) = \frac{W^*(n)}{W(n)}$
  - Efficiency: $E(n) = \frac{W^*(n)}{C(n)} = \frac{S(n)}{p(n)}$, $0 \leq E(n) \leq 1$

    → Cost optimal $\Rightarrow E(n) = 1$ A difficult case

- Tradeoff between time efficiency & cost efficiency
  - To make faster → use more processors, then cost ↗
  - To decrease cost → use less processors, then time ↗

PRAM – Goodness Criteria

- Minimum finding

$$W(n) = \log_2 n \in \theta(\log n)$$

$$S(n) = \frac{n-1}{\log_2 n} \in \theta\left(\frac{n}{\log n}\right)$$

$$C(n) = \frac{n}{2} * \log_2 n \in \theta(n \log n)$$

$$E(n) = \frac{n-1}{(n/2)\log_2 n} = \frac{2(n-1)}{n \log_2 n}$$

```
function MinEREW (L[0:n-1])
Model:   EREW PRAM with p=n/2 processors
Input:   L[0:n-1] (a list of size n, n=2^k)
Output:  minimum of the elements in L

  for i ← 1 to log2n do
    for 0 ≤ j ≤ n/2^i − 1 do in parallel
      if L[2j+1] > L[2j] then
        L[j] ← L[2j]
      else
        L[j] ← L[2j+1]
      endif
    end in parallel
  endfor
  return (L[0])
end
```

- Time efficient? ✓
- Cost efficient? ✓
- Cost optimal? ✗

e.g. n=1000 ⇒ 100 times faster

PRAM – Goodness Criteria

- Minimum finding

$$W(n) = \log_2 n \in \theta(\log n)$$

$$S(n) = \frac{n-1}{\log_2 n} \in \theta\left(\frac{n}{\log n}\right)$$

$$C(n) = \frac{n}{2} * \log_2 n \in \theta(n \log n)$$

$$E(n) = \frac{n-1}{(n/2)\log_2 n} = \frac{2(n-1)}{n \log_2 n}$$

```
function MinEREW (L[0:n-1])
Model:  EREW PRAM with p=n/2 processors
Input:  L[0:n-1] (a list of size n, n=2^k)
Output: minimum of the elements in L

  for i ← 1 to log2n do
    for 0 ≤ j ≤ n/2^i − 1 do in parallel
      if L[2j+1] > L[2j] then
        L[j] ← L[2j]
      else
        L[j] ← L[2j+1]
      endif
    end in parallel
  endfor
  return (L[0])
end
```

- Cost optimal? ✗

Example: n=1000 $\Rightarrow$

Capacity of algorithm = (500 processors) * (10 steps)
= 5,000 basic operations

Number of basic operations executed = 500 + 250 + … + 1
= 1,000 basic operations

PRAM – Goodness Criteria

- Minimum finding

$$W(n) = \log_2 n \in \theta(\log n)$$

$$S(n) = \frac{n-1}{\log_2 n} \in \theta\left(\frac{n}{\log n}\right)$$

$$C(n) = \frac{n}{2} * \log_2 n \in \theta(n \log n)$$

$$E(n) = \frac{n-1}{(n/2)\log_2 n} = \frac{2(n-1)}{n \log_2 n}$$

```
function MinEREW (L[0:n-1])
Model:  EREW PRAM with p=n/2 processors
Input:  L[0:n-1] (a list of size n, n=2^k)
Output: minimum of the elements in L

  for i ← 1 to log2n do
    for 0 ≤ j ≤ n/2^i − 1 do in parallel
      if L[2j+1] > L[2j] then
        L[j] ← L[2j]
      else
        L[j] ← L[2j+1]
      endif
    end in parallel
  endfor
  return (L[0])
end
```

  - Example:

n	Speedup	Efficiency
1,000	100	0.2
1,000,000	50,000	0.1

  - As n ↗ : speedup ↗, but efficiency ↙ ⟹ A conflict?

PRAM – Cost Optimal Minimum Finding

- Can we develop a cost optimal algorithm?
  - $C(n) = p(n) * W(n) \quad \rightarrow \quad \theta(n)$

    Reduce either $p(n)$ or $W(n)$

  - Keep $p(n)$ fixed, reduce $W(n)$

    $\frac{n}{2} \in \theta(n)$

    must be $\theta(1)$

PRAM – Cost Optimal Minimum Finding

- Can we develop a cost optimal algorithm?
  - $C(n) = p(n) * W(n) \quad \rightarrow \quad \theta(n)$

    Reduce either $p(n)$ or $W(n)$

  - Keep $W(n)$ fixed, reduce $p(n)$

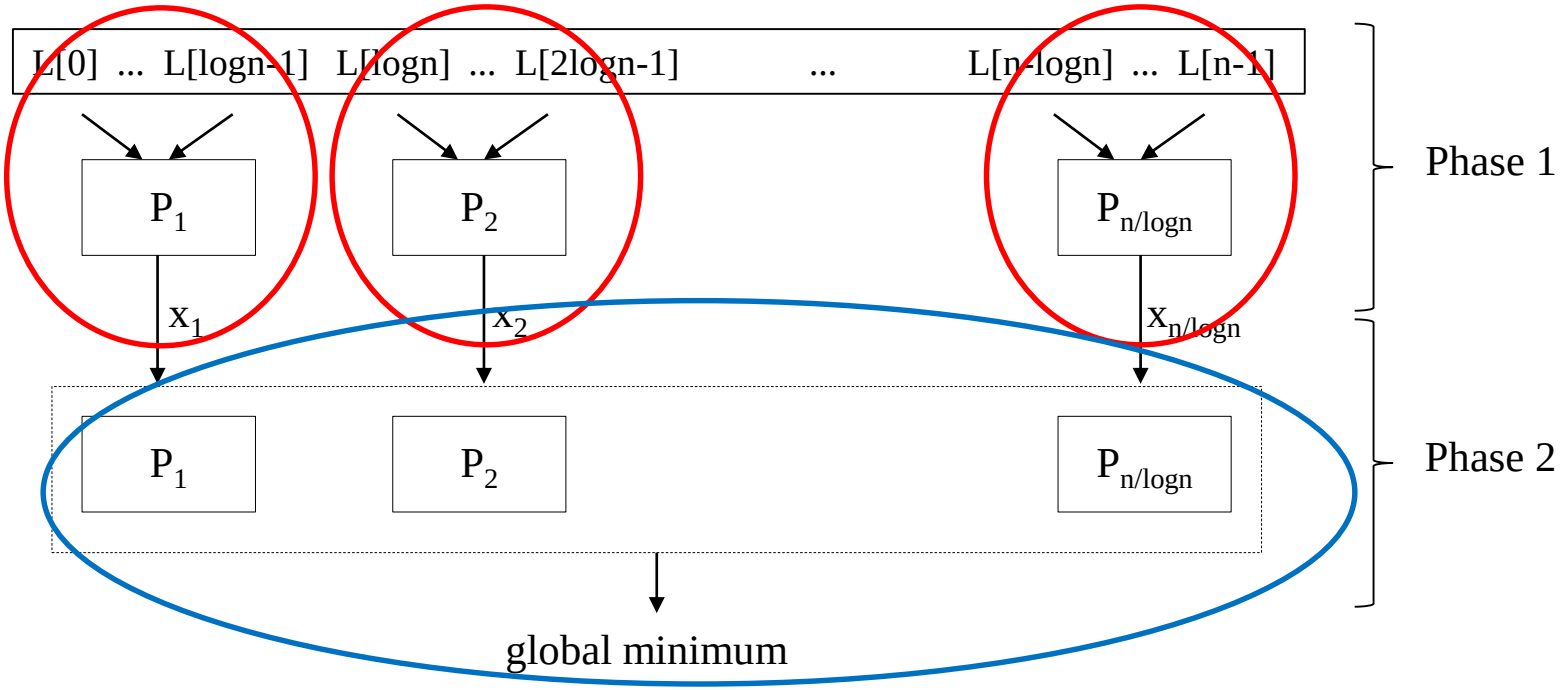
    $\log_2 n \in \theta(\log n)$

    must be $\theta\left(\frac{n}{\log n}\right)$

Any idea?

PRAM – Cost Optimal Minimum Finding

- Can we develop a cost optimal algorithm?



- Phase 1: Each processor finds minimum of $\log n$ items sequentially
- Phase 2: MinEREW algorithm with data size $n/\log n$ & processor number $n/\log n$

PRAM – Cost Optimal Minimum Finding

- Can we develop a cost optimal algorithm?

```
function MinEREW2 (L[0:n-1])
Model:   EREW PRAM with p=n/log2n processors
Input:   L[0:n-1] (a list of size n)
Output:  minimum of the elements in L

  for 0 ≤ i ≤ n/log2 n − 1 do in parallel
    Temp[i] ← L[i * log2n]
    for j ← 1 to log2n-1 do
      if Temp[i] > L[i*log2n+j] then
        Temp[i] ← L
      endif
    endfor
  end in parallel

  return (MinEREW (Temp[0:n/log2 n − 1]))
end
```

Assume $\frac{n}{\log_2 n}$ is integer

Temp[i] : minimum of
$L[i * \log_2 n : (i+1) * \log_2 n - 1]$

Phase 1

$\log_2 n - 1$

$$W(n) = 2\log_2 n - \log_2 \log_2 n - 1 \in \theta(\log n)$$

$$C(n) = \left(\frac{n}{\log_2 n}\right) * (2\log_2 n - \log_2 \log_2 n - 1) \in \theta(n)$$

$\Rightarrow$ Cost optimal

$$\log_2 \frac{n}{\log_2 n} = \log_2 n - \log_2 \log_2 n$$

PRAM – Searching

- Another searching algorithm
  - Searching on a sorted list
  - CRCW PRAM
  - Scalable algorithm

    $p(n)$ is fixed

    Algorithm can execute on any data size n

PRAM – Searching

function SearchOrderedListCRCW (L[0:n-1], X, Found)
Model: CRCW PRAM with p processors
Input: L[0:n-1] (a list of size n)
X (a search element)
Output: Found (a boolean variable. True if X∈L, false otherwise)

Found ← false

for 1 ≤ i ≤ p **do in parallel**

call BinarySearch ($L\left[\frac{(i-1)n}{p} : \frac{in}{p} - 1\right]$, X, Found)

end in parallel
end

- Involves CR?
- Involves CW?
  Which CW mechanism?

PRAM – Searching

- Analysis

$$W(n) = \log_2 \frac{n}{p}$$

$$C(n) = p * \log_2 \frac{n}{p}$$

function SearchOrderedListCRCW (L[0:n-1], X, Found)
Model: CRCW PRAM with p processors
Input: L[0:n-1] (a list of size n)
X (a search element)
Output: Found (a boolean variable. True if X∈L, false otherwise)

Found ← false

for 1 ≤ i ≤ p **do in parallel**

call BinarySearch $(L\left[\frac{(i-1)n}{p} : \frac{in}{p} - 1\right]$, X, Found)

end in parallel
end

- One extreme: To minimize time complexity, $p(n) = ???$

$$p(n) = n \Rightarrow W(n) = \log_2 \frac{n}{n} = 1 \in \theta(1) \quad , \quad \text{but } C(n) = n \in \theta(n)$$

- Other extreme: To minimize cost, $p(n) = ???$

$$p(n) = 1 \Rightarrow C(n) = \log_2 n \in \theta(\log n) \quad , \quad \text{but } W(n) = \log_2 \frac{n}{1} \in$$

Time efficient? ✔
Cost efficient? ✘
Cost optimal? ✘

- Another case:

$$p(n) = \log_2 n \Rightarrow W(n) = \log_2 \frac{n}{\log_2 n} = \log_2 n - \log_2 \log_2 n \in \theta(\log n)$$

$$C(n) = \log_2 n * \log_2 \frac{n}{\log_2 n} \in \theta(log^2 n)$$

Study: Odd-Even Mergesort